

Trabajo Práctico — Semana 4

Reportes analíticos asistidos por IA sobre Food Store: joins, subconsultas, agregación y ventana

Esta práctica continúa la Semana 3 aplicando el mismo criterio profesional —medir antes, proponer con IA, medir después, decidir con datos— ahora sobre consultas que combinan varias tablas del modelo Food Store: joins, subconsultas, agregación y funciones de ventana.

Objetivos de la práctica

- Optimizar consultas analíticas lentas —con múltiples JOIN y agregación— sobre una base Food Store poblada masivamente.
- Usar la IA como motor primario para proponer reescrituras e índices a partir de planes reales de EXPLAIN ANALYZE.
- Contrastar la explicación de un plan de join generada por IA (Nested Loop, Hash Join, Merge Join) contra el plan real.
- Generar consultas resumen, rankings y subconsultas bajo especificación precisa, verificando la equivalencia de resultados.
- Documentar cada decisión en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store poblada masivamente, tal como quedó al cierre de la práctica de la Semana 3 (o repetir la carga si se trabaja sobre una copia nueva).
- Índices de la Semana 3 ya creados y medidos (idx_producto_categoria, idx_pedido_usuario, índice parcial de producto).
- Acceso a psql / DBeaver / pgAdmin y a la IA de la cátedra (OpenCode / Kiro).

Parte 1 — Laboratorio: consultas analíticas lentas

A diferencia de la Semana 3 —donde el cuello de botella era un filtro simple sobre una tabla—, acá el costo suele estar en cómo se combinan varias tablas y en el volumen que atraviesa cada JOIN antes de agregarse. Elegir consultas reales de queries.sql que crucen al menos tres tablas del modelo.

1.1 Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.
3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.

Consulta 1:

Propuesta A1 — Índice parcial sobre pedido

```
CREATE INDEX idx_pedido_conf_ter ON pedido (id)
WHERE estado IN ('CONFIRMADO', 'TERMINADO');
```

Justificación (nodo concreto): El nodo Seq Scan on pedido p (cost 0.00..4370.00) lee 200.000 filas y el Filter elimina 99.690 (Rows Removed by Filter: 99690) para alimentar el Hash con solo 100.310. El índice parcial materializa ese filtro: contiene exactamente de antemano los id que cumplen la condición (el predicado es la definición del índice, así que es index-only, sin tocar el heap).

Propuesta A2 — Índice de cobertura sobre detalle_pedido

```
CREATE INDEX idx_dp_cubrimiento ON detalle_pedido (pedido_id)
INCLUDE (producto_id, cantidad);
```

Justificación (nodo concreto): El Seq Scan on detalle_pedido dp (cost 0.00..4273.00, Buffers: shared hit=2273) lee el heap completo, pero la consulta solo usa tres columnas de esa tabla (pedido_id para el join, producto_id para el join 2 y el group, cantidad para el SUM). Con el índice de cobertura el planner puede servir esas tres columnas desde el índice sin abrir el heap.

Propuesta B — Reescritura: agregar antes de tocar producto

```
SELECT prod.nombre, v.unidades_vendidas
FROM (
    SELECT dp.producto_id, SUM(dp.cantidad) AS unidades_vendidas
    FROM detalle_pedido dp
    JOIN pedido p ON dp.pedido_id = p.id
    WHERE p.estado IN ('CONFIRMADO', 'TERMINADO')
    GROUP BY dp.producto_id
) v
JOIN producto prod ON prod.id = v.producto_id
ORDER BY v.unidades_vendidas DESC;
```

Justificación (nodos concretos): El plan actual agrega DESPUÉS de cruzar con producto: el Group Key: prod.nombre obliga a que el nombre viaje por Hash Join (dp.pedido_id = p.id) y por Hash Join (dp.producto_id = prod.id) (el plan lo refleja en width=25 del segundo join), y el HashAggregate procesa las 100.310 filas ya ensambladas. La reescritura mueve el agregado antes del join con producto

Consulta 2:

Propuesta 2 — Índice cubridor en detalle_pedido para evitar el Seq Scan

El problema concreto en el plan:

El nodo más caro es el Parallel Seq Scan on detalle_pedido dp: 2,273 buffers para leer 100,000 filas. Este scan es el que alimenta el primer hash join, y es la mayor fuente de I/O lógica del plan. El optimizador elige Seq Scan porque necesita pedido_id, producto_id (para el join con producto, si se mantiene) y cantidad (para la agregación) — no hay índice que cubra estos tres campos.

Propuesta concreta:

```
CREATE INDEX idx_detalle_pedido_covering
ON detalle_pedido (pedido_id, producto_id, cantidad);
```

El Seq Scan se reemplaza por un Index Only Scan porque el índice contiene exactamente las columnas que necesita la consulta (pedido_id para el join condition, cantidad para la agregación, y producto_id solo si se mantiene el join con producto). PostgreSQL puede responder toda la consulta leyendo solo el B-tree del índice sin tocar la tabla heap.

5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
Productos más vendidos (volumen de alimentos)	Hash Join	Se aplicó índice parcial sobre la tabla pedido para evitar filtrar todo el heap y se reescribió la consulta para agrupar el total de ventas antes del cruce con la tabla producto	Hash Join	El tiempo de ejecución se redujo de 439.714 ms a 342.212 ms. Se eliminó el Seq Scan en pedido sustituyéndolo por un Index Only Scan, y la agregación temprana procesó un volumen menor en el segundo Hash Join
Volumen de artículos según forma de pago	Hash Join (Parallel Hash Join)	Creación de índice de cobertura en detalle_pedido (pedido_id, producto_id, cantidad) sugerido por el agente	Hash Join (Parallel Hash Join)	El tiempo de ejecución se redujo de 318.983 ms a 242.542 ms. Se documenta que el optimizador decidió no utilizar el índice y mantuvo el Seq Scan, ya que al no existir filtros, el escaneo secuencial masivo resultó ser la estrategia de menor costo para el motor

Criterio de aceptación

Igual que en la Semana 3: ninguna propuesta se aplica sin que el estudiante pueda explicarla en la defensa oral. Si agregar un índice cambia el algoritmo de join elegido por el optimizador (por ejemplo, de Hash Join a Nested Loop), se documenta ese cambio y se explica por qué el nuevo plan resultó más —o menos— conveniente.

Parte 2 — Lectura crítica de planes de join interpretados por IA

Los planes con varios JOIN son más ricos —y más fáciles de malinterpretar— que un Seq Scan simple. Esta parte entrena la lectura crítica sobre ese tipo de plan.

1. Tomar uno de los planes reales obtenidos en la Parte 1, con al menos dos nodos de join.
2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin más contexto que el texto del plan.
3. Contrastar la explicación contra el plan real: verificar en particular que la IA identifique correctamente qué tabla es la “externa” y cuál la “interna” en cada Nested Loop, y que no confunda el costo estimado de un nodo intermedio con el tiempo total de la consulta.
4. Documentar los hallazgos en la tabla siguiente

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
"El plan comienza ejecutando un Nested Loop, donde la tabla pedido actúa como la tabla interna (inner) porque es la primera que se escanea en el árbol."	No	En PostgreSQL, el nodo superior en un Nested Loop (el que se lee primero para iniciar las iteraciones) es siempre la tabla externa (outer), no la interna. La tabla interna es la que se escanea repetidamente por cada fila de la externa
"El nodo de Hash Join intermedio tiene un costo de 4273.00 , lo que significa que el motor tardó exactamente 4273 milisegundos en combinar ambas tablas."	No	La IA está confundiendo la estimación del optimizador con el tiempo de ejecución real. El valor cost=4273.00 es una unidad abstracta de esfuerzo, mientras que el tiempo total se debe observar en la métrica actual time=...ms generada por EXPLAIN ANALYZE
"Para el segundo cruce, se emplea un Hash Join donde la tabla producto funciona como la tabla interna, ya que se utiliza para construir la tabla hash en memoria."	Si	El plan real evidencia un nodo Hash posicionado directamente por encima del escaneo (Seq Scan o Index Scan) de la tabla producto . Esto confirma que sus registros se cargan en memoria para ser consultados, actuando correctamente como la relación interna.

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

Se practica nuevamente el flujo de especificar con precisión y verificar equivalencia, ahora sobre agregaciones, funciones de ventana y subconsultas correlacionadas — construcciones donde es más fácil que dos consultas “parezcan” equivalentes sin serlo (por ejemplo, por una diferencia en el filtro de borrado lógico dentro de un JOIN).

1. Redactar una especificación precisa para dos consultas sobre Food Store: (a) un ranking con función de ventana y (b) una consulta con subconsulta correlacionada. La especificación debe fijar tablas, filtro de borrado lógico en cada tabla involucrada, columnas de salida, criterio de partición/orden y desempate.

Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada producto vigente (borrado lógico: eliminado = false), su nombre y el total de unidades vendidas acumuladas en pedidos que estén en estado 'CONFIRMADO' o 'TERMINADO'. Además, la consulta debe incluir el puesto del producto en un ranking ordenado de mayor a menor volumen de ventas. En caso de empate en la cantidad, deben compartir el mismo puesto (sin colapsar filas ni dejar huecos en la numeración). No uses SELECT *.

2. Pedirle a la IA que genere el SQL a partir de la especificación.

```
SELECT
  prod.nombre,
  SUM(dp.cantidad) AS unidades_vendidas,
  DENSE_RANK() OVER (ORDER BY SUM(dp.cantidad) DESC) AS puesto
FROM producto prod
JOIN detalle_pedido dp ON prod.id = dp.producto_id
JOIN pedido p ON dp.pedido_id = p.id
WHERE prod.eliminado = false
  AND p.estado IN ('CONFIRMADO', 'TERMINADO')
GROUP BY prod.id, prod.nombre;
```

3. Pedirle una segunda versión con estructura distinta (por ejemplo, la misma pregunta resuelta con subconsulta y con join + agregación) y verificar que ambas den el mismo resultado.

```
SELECT
  v_principal.nombre,
  v_principal.unidades_vendidas,
  (
    SELECT COUNT(DISTINCT SUM(dp2.cantidad)) + 1
```

```
FROM producto prod2

JOIN detalle_pedido dp2 ON prod2.id = dp2.producto_id

JOIN pedido p2 ON dp2.pedido_id = p2.id

WHERE prod2.eliminado = false

AND p2.estado IN ('CONFIRMADO', 'TERMINADO')

GROUP BY prod2.id

HAVING SUM(dp2.cantidad) > v_principal.unidades_vendidas

) AS puesto

FROM (

SELECT prod.id, prod.nombre, SUM(dp.cantidad) AS unidades_vendidas

FROM producto prod

JOIN detalle_pedido dp ON prod.id = dp.producto_id

JOIN pedido p ON dp.pedido_id = p.id

WHERE prod.eliminado = false

AND p.estado IN ('CONFIRMADO', 'TERMINADO')

GROUP BY prod.id, prod.nombre

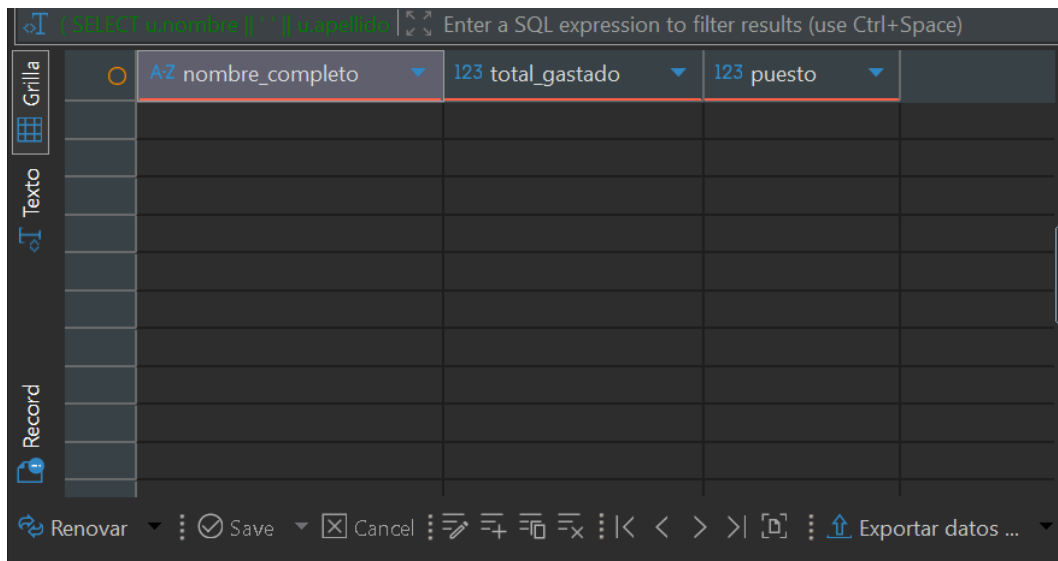
) AS v_principal;
```

4. Verificar la equivalencia con EXCEPT o comparando conteos y una muestra de filas, igual que en la Semana

```
-- Ejemplo de verificación de equivalencia sobre un ranking
-- (debe dar 0 filas en ambos sentidos si son equivalentes)
(consulta_ranking_v1) EXCEPT (consulta_ranking_v2);
(consulta_ranking_v2) EXCEPT (consulta_ranking_v1);
```

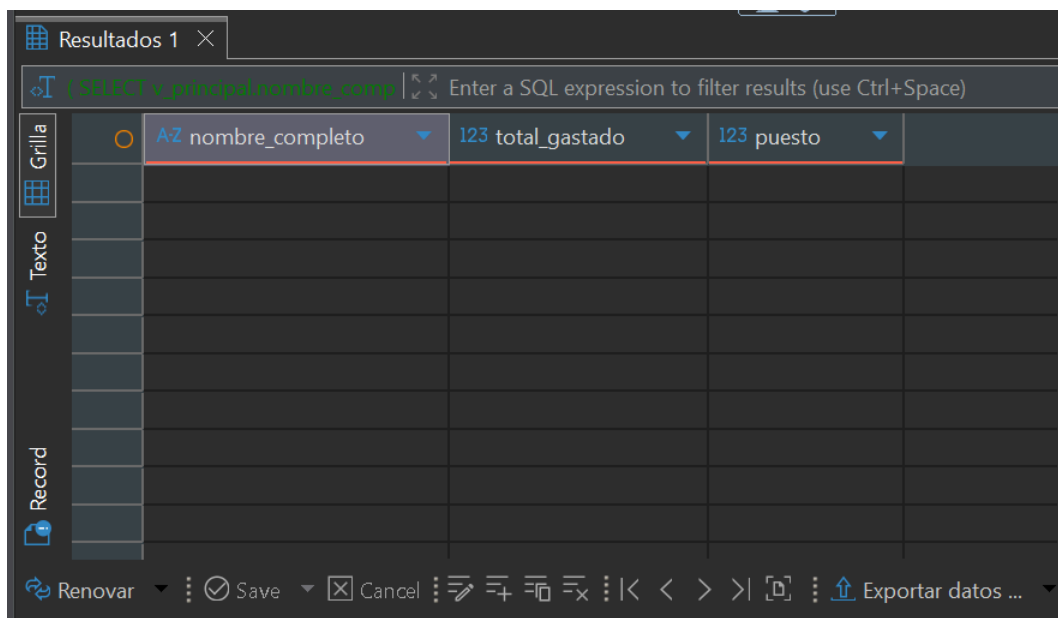
Spec de ejemplo

«Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada usuario vigente con al menos un pedido vigente, su nombre completo, el total gastado (suma de pedido.total en pedidos no eliminados) y su puesto en un ranking de mayor a menor gasto, sin colapsar filas. En caso de empate, deben compartir el mismo puesto. No uses SELECT *.»



The screenshot shows a SQL query editor with a query bar at the top containing the text: `( SELECT u.nombre || ' ' || u.apellido`. Below the query bar is a table with three columns: `nombre_completo`, `total_gastado`, and `puesto`. The table is currently empty. The interface includes a sidebar with icons for 'Grilla', 'Texto', and 'Record', and a bottom toolbar with buttons for 'Renovar', 'Save', 'Cancel', and 'Exportar datos ...'.

nombre_completo	total_gastado	puesto
-----------------	---------------	--------



The screenshot shows a SQL query editor with a query bar at the top containing the text: `( SELECT v.principal.nombre_comp`. Below the query bar is a table with three columns: `nombre_completo`, `total_gastado`, and `puesto`. The table is currently empty. The interface includes a sidebar with icons for 'Grilla', 'Texto', and 'Record', and a bottom toolbar with buttons for 'Renovar', 'Save', 'Cancel', and 'Exportar datos ...'.

nombre_completo	total_gastado	puesto
-----------------	---------------	--------

Parte 4 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta analítica lenta —con al menos dos JOIN y una agregación— y compiten por el mejor plan, con la IA como asistente.

- Consigna común: una consulta de facturación o ranking, igual para todos los equipos, sobre la base masiva compartida.

Consulta:

```
SELECT pr.id, pr.nombre
FROM   producto pr
LEFT   JOIN detalle_pedido dp
        ON dp.producto_id = pr.id AND dp.eliminado = FALSE
WHERE  pr.eliminado = FALSE
        AND dp.id IS NULL
ORDER BY pr.id;
```

- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar en el plan real.

Consulta reescrita:

```
SELECT id, nombre
FROM producto
WHERE eliminado = FALSE
```

EXCEPT

```
SELECT pr.id, pr.nombre
FROM producto pr
JOIN detalle_pedido dp ON dp.producto_id = pr.id
WHERE pr.eliminado = FALSE
AND dp.eliminado = FALSE
```

ORDER BY id;

- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE).
- Se documenta en la bitácora toda propuesta de la IA que no funcionó, no solo la que se terminó usando.

Otras consultas propuestas:

```
SELECT id, nombre
FROM producto
WHERE eliminado = FALSE
AND id NOT IN (
  SELECT producto_id
  FROM detalle_pedido
  WHERE eliminado = FALSE
  AND producto_id IS NOT NULL
)
ORDER BY id;
```

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
...	Teoria de conjuntos	P:0.239 ms E:0.112 ms	P:0.126 ms E:0.054 ms	P:0.113 ms E:0.058

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica. Completar una fila por cada uso relevante de IA.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode (gemini flash 3.6)	optimización de consulta	[CONSULTA ORIGINAL] Necesito que optimices esta consulta para que tenga un menor tiempo de planificación y ejecución. Dame varias opciones	Se aceptó la primer opción que dio ya que fue la que más diferencia tenía en los tiempos comparado a la consulta original
opencode (gemini)	redactar especificaciones precisas para consultas en la food store	necesito que me hagas consultas para mi food store 1-un ranking con función de ventana 2-una consulta con subconsulta correlacionada	Se aceptaron ambas ya que cumplían con los requisitos
opencode (gemini)	Pedir la misma consulta pero con diferente estructura	Necesito que a las consultas que creaste les cambies la estructura pero sin modificar su comportamiento	se aceptaron ya que dieron los mismos resultados que las originales

Entregables

- Tabla comparativa de la Parte 1, con capturas o texto de los planes antes/después, incluyendo el algoritmo de join identificado en cada caso.
- Tabla de lectura crítica de la Parte 2.
- Las dos consultas de la Parte 3 (spec + SQL generado + verificación de equivalencia).
- Registro de la competencia de la Parte 4.
- Declaración de Uso de IA (DUIA) completa.
- Defensa oral de cualquier script o índice de la entrega.

Criterios de evaluación

Dimensión	Qué se evalúa	Peso
Medición	EXPLAIN ANALYZE correcto y completo, antes y después, identificando el algoritmo de join	30%

Dimensión	Qué se evalúa	Peso
Criterio de aceptación	Las propuestas de la IA se validan contra el plan real, no se aplican a ciegas	25%
Lectura crítica	Se detectan imprecisiones reales sobre planes con varios JOIN	20%
Equivalencia verificada	Las consultas alternativas —incluyendo rankings— se comprueban formalmente equivalentes	15%
Bitácora y defensa	DUIA completa y capacidad de explicar cada decisión	10%